

人工智能化学计算模拟

第一次作业

盛泓睿 23307110041

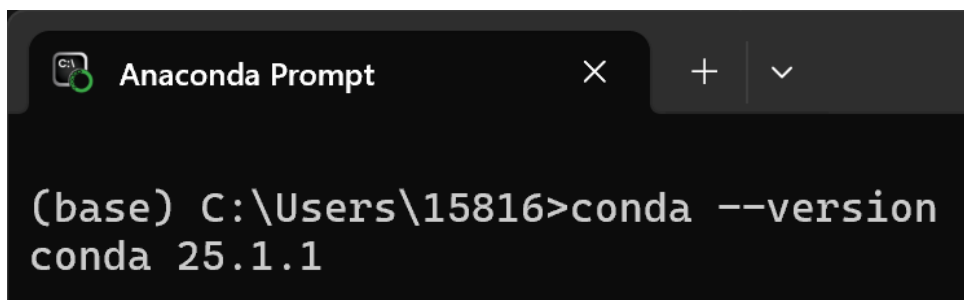
1 作业内容概述

本次作业旨在完成本课程所需的基础开发环境配置，包括 Anaconda 虚拟环境管理工具的安装、VSCode 环境的配置。同时，通过调用大语言模型辅助编程辅助研究神经网络，并实现在本地环境中的代码执行。

2 环境安装与配置

2.1 Conda 环境安装

首先安装 Anaconda。通过终端输入相关命令验证安装状态，确保 Conda 能够正常管理 Python 虚拟环境。



```
(base) C:\Users\15816>conda --version
conda 25.1.1
```

图 1: Conda 安装验证

2.2 VSCode 开发环境

配置了 Visual Studio Code，并安装了 Python、Jupyter 以及相关的 AI 编程插件（GitHub Copilot 和 Continue）。

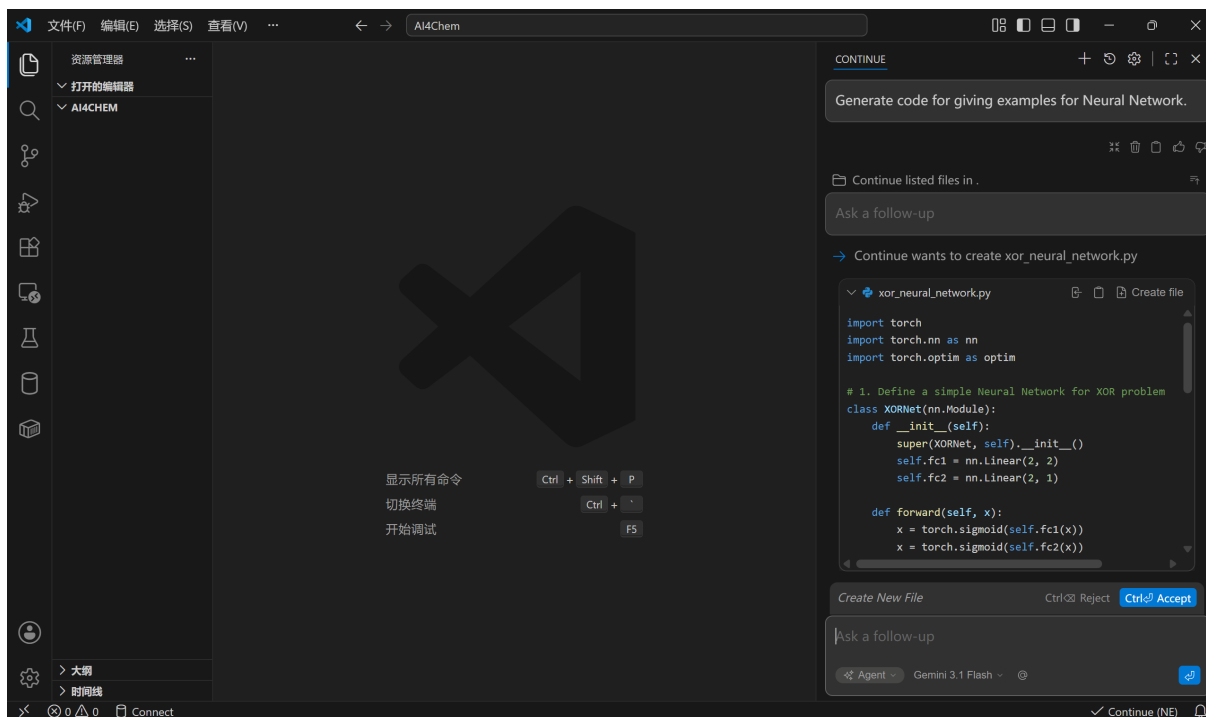


图 2: VSCode 开发环境配置

3 大语言模型辅助神经网络研究

本次作业分别使用了 GitHub Copilot 和 Continue 插件（配置 Gemini 3.1 Flash 模型）进行神经网络代码的辅助生成。

3.1 交互过程

通过向模型描述需求，获取了基础代码架构。

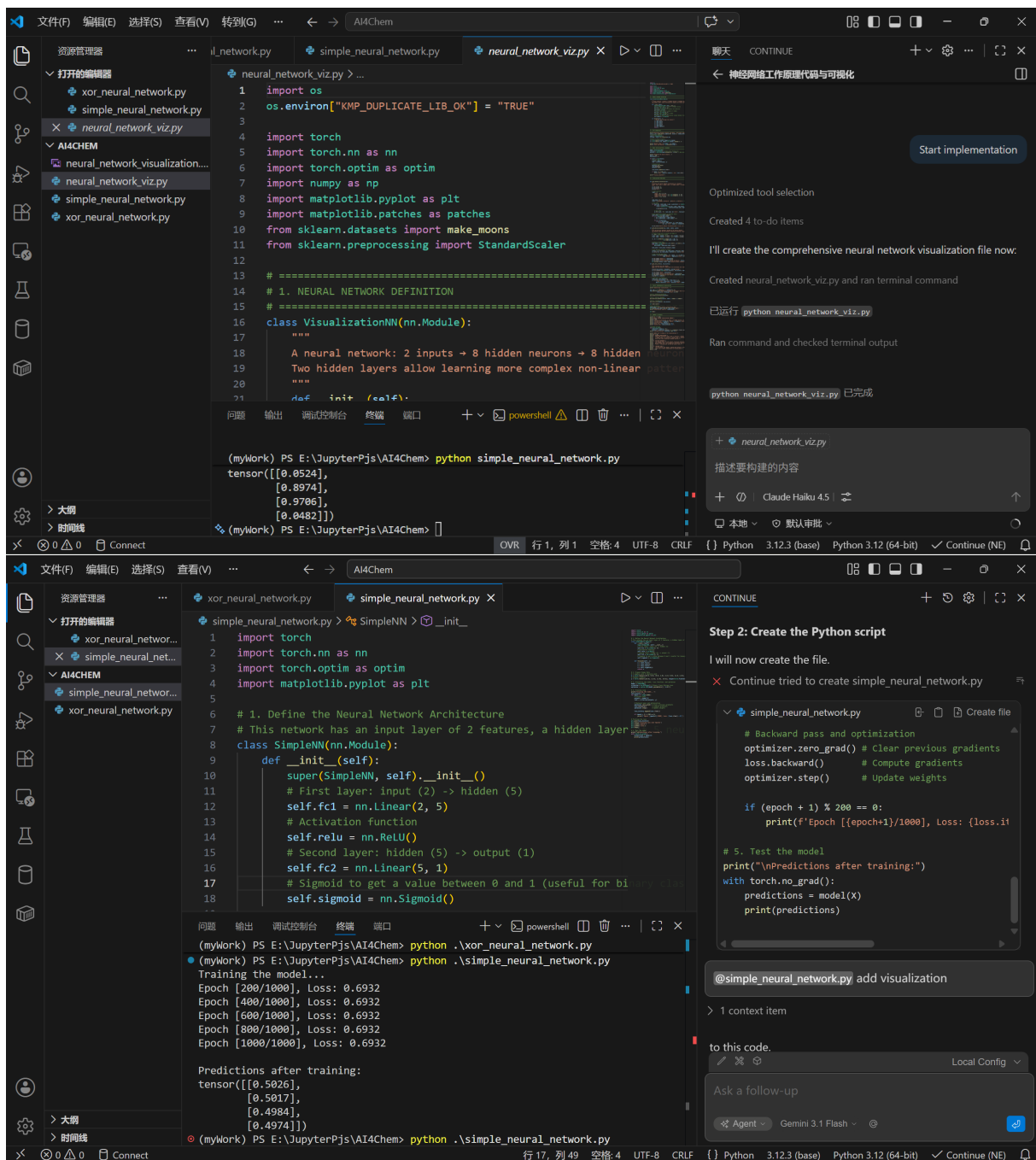


图 3: 大语言模型辅助交互过程

3.2 生成的神经网络代码

以下为 LLM 生成的简单神经网络示例代码。

```
1 import os
2 os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
3 import torch
```

```

4 import torch.nn as nn
5 import torch.optim as optim
6 import matplotlib.pyplot as plt
7
8 # 1. Define the Neural Network Architecture
9 # This network has an input layer of 2 features, a hidden layer of 5
   neurons, and 1 output neuron.
10 class SimpleNN(nn.Module):
11     def __init__(self):
12         super(SimpleNN, self).__init__()
13         # First layer: input (2) -> hidden (5)
14         self.fc1 = nn.Linear(2, 5)
15         # Activation function
16         self.relu = nn.ReLU()
17         # Second layer: hidden (5) -> output (1)
18         self.fc2 = nn.Linear(5, 1)
19         # Sigmoid to get a value between 0 and 1 (useful for binary
           classification)
20         self.sigmoid = nn.Sigmoid()
21
22     def forward(self, x):
23         x = self.fc1(x)
24         x = self.relu(x)
25         x = self.fc2(x)
26         x = self.sigmoid(x)
27         return x
28
29 # 2. Prepare Dummy Data
30 # 4 samples with 2 features each
31 X = torch.tensor([[0.0, 0.0], [0.0, 1.0], [1.0, 0.0], [1.0, 1.0]], dtype=
   torch.float32)
32 # Targets (labels)
33 y = torch.tensor([[0.0], [1.0], [1.0], [0.0]], dtype=torch.float32)
34
35 # 3. Initialize the model, loss function, and optimizer
36 model = SimpleNN()
37 criterion = nn.BCELoss() # Binary Cross Entropy Loss
38 optimizer = optim.SGD(model.parameters(), lr=0.1)
39
40 # 4. Training Loop
41 print("Training the model...")
42 loss_history = []
43 for epoch in range(1000):

```

```

44     # Forward pass
45     outputs = model(X)
46     loss = criterion(outputs, y)
47
48     # Backward pass and optimization
49     optimizer.zero_grad() # Clear previous gradients
50     loss.backward()        # Compute gradients
51     optimizer.step()       # Update weights
52
53     loss_history.append(loss.item())
54
55     if (epoch + 1) % 200 == 0:
56         print(f'Epoch_{epoch+1}/1000, Loss:{loss.item():.4f}')
57
58 # Plotting the loss
59 plt.plot(loss_history)
60 plt.title('Training Loss over Epochs')
61 plt.xlabel('Epoch')
62 plt.ylabel('Loss')
63 plt.show()
64
65 # 5. Test the model
66 print("\nPredictions after training:")
67 with torch.no_grad():
68     predictions = model(X)
69     print(predictions)

```

Listing 1: 简单神经网络代码示例

4 程序执行与结果验证

在 VSCode 中执行代码，获得可视化输出。

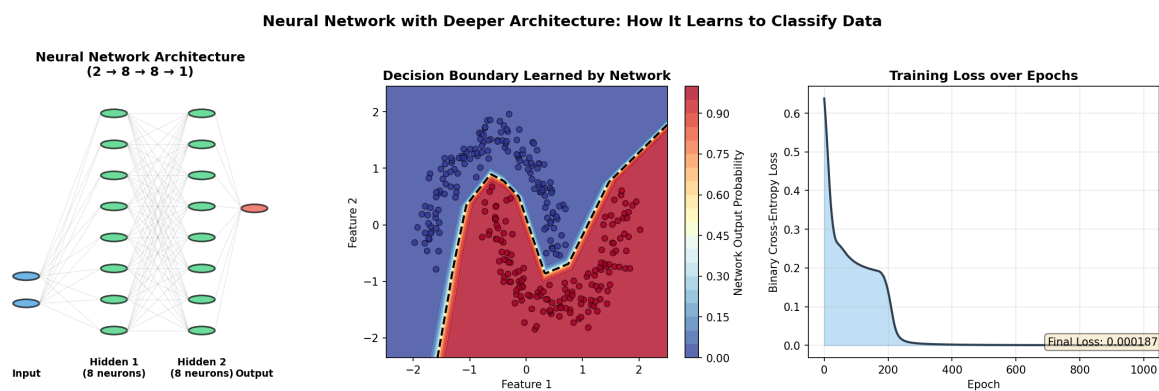


图 4: 代码执行过程及结果截图

5 总结

本次作业完成了从环境搭建到代码实践的过程。在 VSCode 中整合 GitHub Copilot 与 Continue (Gemini API) 提升了代码编写效率。